

```

/* =====
TRIGGERS IN ORACLE PL/SQL

Topics:
- DML Triggers
  * BEFORE INSERT
  * AFTER INSERT
  * BEFORE UPDATE
  * AFTER UPDATE
  * BEFORE DELETE
  * AFTER DELETE

- Row Level Trigger
- Statement Level Trigger
- OLD and NEW Values

Labs:
- Audit Log Trigger
- Salary History Trigger
- Customer Activity Tracking
===== */

SET SERVEROUTPUT ON SIZE UNLIMITED;

/* =====
1. DROP OLD OBJECTS
===== */

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE customer_activity_logs';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE salary_history';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE audit_logs';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE customers';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;

```

```

/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE employees';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

/* =====
2. CREATE MAIN TABLES
===== */

CREATE TABLE employees
(
    employee_id    NUMBER PRIMARY KEY,
    employee_name  VARCHAR2(100),
    department     VARCHAR2(50),
    salary         NUMBER,
    status         VARCHAR2(20),
    created_date   DATE,
    updated_date   DATE
);

CREATE TABLE customers
(
    customer_id    NUMBER PRIMARY KEY,
    customer_name  VARCHAR2(100),
    phone         VARCHAR2(30),
    city          VARCHAR2(50),
    status         VARCHAR2(20),
    created_date   DATE,
    updated_date   DATE
);

/* =====
3. CREATE LOG TABLES
===== */

CREATE TABLE audit_logs
(
    log_id         NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    table_name     VARCHAR2(50),
    action_name    VARCHAR2(30),
    record_id      NUMBER,
    old_value      VARCHAR2(4000),
    new_value      VARCHAR2(4000),
    action_date    DATE DEFAULT SYSDATE,
    action_user    VARCHAR2(100)
);

CREATE TABLE salary_history
(
    history_id     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

```

```

        employee_id      NUMBER,
        old_salary        NUMBER,
        new_salary        NUMBER,
        changed_amount    NUMBER,
        changed_date      DATE DEFAULT SYSDATE,
        changed_by        VARCHAR2(100)
    );

CREATE TABLE customer_activity_logs
(
    activity_id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id          NUMBER,
    activity_type        VARCHAR2(50),
    description          VARCHAR2(4000),
    activity_date        DATE DEFAULT SYSDATE,
    activity_user        VARCHAR2(100)
);

/* =====
4. BEFORE INSERT TRIGGER

Purpose:
- Set default created_date before inserting employee
- Set default status if status is NULL

:NEW means new value being inserted.
===== */

CREATE OR REPLACE TRIGGER trg_emp_before_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF :NEW.created_date IS NULL THEN
        :NEW.created_date := SYSDATE;
    END IF;

    IF :NEW.status IS NULL THEN
        :NEW.status := 'ACTIVE';
    END IF;

    DBMS_OUTPUT.PUT_LINE('BEFORE INSERT Trigger: Employee data
prepared. ');
END;
/

```

=====

Testing before insert trigger "trg_emp_before_insert"

```
SQL> INSERT INTO employees
(employee_id, employee_name, department, salary, status)
VALUES
(1, 'Aung Aung', 'IT', 900000, NULL);
  2      3      4 BEFORE INSERT Trigger: Employee data prepared.

1 row created.

SQL> SELECT * FROM employees;

EMPLOYEE_ID
-----
EMPLOYEE_NAME
-----
DEPARTMENT                                     SALARY
-----
STATUS          CREATED_D  UPDATED_D
-----
              1
Aung Aung
IT
ACTIVE          21-JUN-26          900000
```

```
/* =====
5. AFTER INSERT TRIGGER

Purpose:
- Insert audit log after new employee is inserted.
===== */
```

```
CREATE OR REPLACE TRIGGER trg_emp_after_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_logs
    (
        table_name,
        action_name,
        record_id,
        old_value,
        new_value,
        action_user
    )
    VALUES
    (
        'EMPLOYEES',
        'INSERT',
        :NEW.employee_id,
        NULL,
        'Name=' || :NEW.employee_name ||
        ', Dept=' || :NEW.department ||
        ', Salary=' || :NEW.salary ||
    )
END;
```

```

        ', Status=' || :NEW.status,
        USER
    );

    DBMS_OUTPUT.PUT_LINE('AFTER INSERT Trigger: Employee audit log
inserted.');
```

=====

After Insert Trigger Testing

```

SQL> INSERT INTO employees
(employee_id, employee_name, department, salary, status)
VALUES
(2, 'Mg Mg', 'HR', 600000, 'ACTIVE');
```

2 3 4 BEFORE INSERT Trigger: Employee data prepared.
AFTER INSERT Trigger: Employee audit log inserted.

1 row created.

```

SQL> SELECT * FROM employees;
```

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
1	Aung Aung	IT	900000	ACTIVE	21-JUN-26	

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
2	Mg Mg	HR	600000	ACTIVE	21-JUN-26	

```
SQL> SELECT * FROM audit_logs;
```

LOG_ID	TABLE_NAME
ACTION_NAME	RECORD_ID
OLD_VALUE	
NEW_VALUE	
ACTION_DA	
ACTION_USER	

1 EMPLOYEES

LOG_ID	TABLE_NAME
ACTION_NAME	RECORD_ID
OLD_VALUE	
NEW_VALUE	
ACTION_DA	
ACTION_USER	

INSERT 2

LOG_ID	TABLE_NAME
ACTION_NAME	RECORD_ID
OLD_VALUE	
NEW_VALUE	
ACTION_DA	
ACTION_USER	
LOG_ID	TABLE_NAME
ACTION_NAME	RECORD_ID
OLD_VALUE	
NEW_VALUE	
ACTION_DA	
ACTION_USER	
Name=Mg Mg, Dept=HR, Salary=600000, Status=ACTIVE	
LOG_ID	TABLE_NAME
ACTION_NAME	RECORD_ID
OLD_VALUE	
NEW_VALUE	
ACTION_DA	
ACTION_USER	
21-JUN-26	
LOG_ID	TABLE_NAME
ACTION_NAME	RECORD_ID
OLD_VALUE	
NEW_VALUE	
ACTION_DA	
ACTION_USER	
SYSTEM	

```

/* =====
INSERT SAMPLE EMPLOYEES
These INSERT statements will fire:
- BEFORE INSERT
- AFTER INSERT
===== */

```

```

SQL> INSERT INTO employees
(employee_id, employee_name, department, salary, status)
VALUES
(3, 'Su Su', 'Finance', 1200000, 'INACTIVE');
  2      3      4 BEFORE INSERT Trigger: Employee data prepared.
AFTER INSERT Trigger: Employee audit log inserted.

```

1 row created.

```
SQL> SELECT * FROM employees;
```

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
1	Aung Aung	IT	900000	ACTIVE	21-JUN-26	

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
2	Mg Mg	HR	600000	ACTIVE	21-JUN-26	

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
3	Su Su	Finance	1200000	INACTIVE	21-JUN-26	

```
COMMIT;
```



```

/* =====
6. BEFORE UPDATE TRIGGER

Purpose:
- Set updated_date before employee update.
- Prevent salary from becoming negative.
===== */

CREATE OR REPLACE TRIGGER trg_emp_before_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be negative.');
```

END IF;

 :NEW.updated_date := SYSDATE;

 DBMS_OUTPUT.PUT_LINE('BEFORE UPDATE Trigger: Employee update validated.');

END;

/

```

/* =====
7. AFTER UPDATE TRIGGER

Purpose:
- Insert audit log after employee update.
- Demonstrate :OLD and :NEW values.
===== */

CREATE OR REPLACE TRIGGER trg_emp_after_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_logs
    (
        table_name,
        action_name,
        record_id,
        old_value,
        new_value,
        action_user
    )
    VALUES
    (
        'EMPLOYEES',
        'UPDATE',
        :NEW.employee_id,
        'Old Salary=' || :OLD.salary ||
        ', Old Status=' || :OLD.status,
        'New Salary=' || :NEW.salary ||
        ', New Status=' || :NEW.status,
        USER
    )

```

```

);

DBMS_OUTPUT.PUT_LINE('AFTER UPDATE Trigger: Employee audit log
inserted.');
```

```

END;
/

```

===

Testing for before update trigger and after update trigger
 OLD and NEW Values Example

```

:OLD.salary = salary before update
:NEW.salary = salary after update
Already demonstrated in trg_emp_after_update.

```

=====

```

SQL> UPDATE employees
SET salary = salary + 100000
WHERE employee_id = 1;
  2      3 BEFORE UPDATE Trigger: Employee update validated.
AFTER UPDATE Trigger: Employee audit log inserted.

1 row updated.
SQL> SELECT * FROM employees;

EMPLOYEE_ID
-----
EMPLOYEE_NAME
-----
DEPARTMENT                                SALARY
-----
STATUS          CREATED_D  UPDATED_D
-----
1
Aung Aung
IT
ACTIVE          21-JUN-26 21-JUN-26
1000000

```

```

SELECT * FROM audit_logs;

```

```

LOG_ID TABLE_NAME
-----
ACTION_NAME          RECORD_ID
-----
OLD_VALUE
-----
NEW_VALUE
-----
ACTION_DA
-----
ACTION_USER
-----
      3 EMPLOYEES
LOG_ID TABLE_NAME
-----
ACTION_NAME          RECORD_ID
-----
OLD_VALUE
-----
NEW_VALUE
-----
ACTION_DA
-----
ACTION_USER
-----
UPDATE              1
LOG_ID TABLE_NAME
-----
ACTION_NAME          RECORD_ID
-----
OLD_VALUE
-----
NEW_VALUE
-----
ACTION_DA
-----
ACTION_USER
-----
Old Salary=900000, Old Status=ACTIVE
LOG_ID TABLE_NAME
-----
ACTION_NAME          RECORD_ID
-----
OLD_VALUE
-----
NEW_VALUE
-----
ACTION_DA
-----
ACTION_USER
-----
New Salary=1000000, New Status=ACTIVE

```

COMMIT;

```

/* =====
8. BEFORE DELETE TRIGGER

Purpose:
- Prevent deleting ACTIVE employees.
===== */

CREATE OR REPLACE TRIGGER trg_emp_before_delete
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    IF :OLD.status = 'ACTIVE' THEN
        RAISE_APPLICATION_ERROR(-20002, 'Cannot delete ACTIVE
employee. ');
    END IF;

    DBMS_OUTPUT.PUT_LINE('BEFORE DELETE Trigger: Employee delete
validated. ');
END;
/

/* =====
9. AFTER DELETE TRIGGER

Purpose:
- Insert audit log after employee delete.
- :OLD means value before delete.
===== */

CREATE OR REPLACE TRIGGER trg_emp_after_delete
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_logs
    (
        table_name,
        action_name,
        record_id,
        old_value,
        new_value,
        action_user
    )
    VALUES
    (
        'EMPLOYEES',
        'DELETE',
        :OLD.employee_id,
        'Name=' || :OLD.employee_name ||
        ', Dept=' || :OLD.department ||
        ', Salary=' || :OLD.salary ||
        ', Status=' || :OLD.status,
        NULL,
        USER
    );

```

```

        DBMS_OUTPUT.PUT_LINE('AFTER DELETE Trigger: Employee audit log
inserted.');
```

```

END;
/
```

```

/* =====
Testing  DELETE EMPLOYEE
This DELETE is allowed because employee 3 is INACTIVE.
===== */
```

```

SQL> DELETE FROM employees
WHERE employee_id = 3;
      2  BEFORE DELETE Trigger: Employee delete validated.
AFTER DELETE Trigger: Employee audit log inserted.

1 row deleted.

COMMIT;
```

```

SQL> SELECT * FROM employees;
```

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
1	Aung Aung	IT	1000000	ACTIVE	21-JUN-26	21-JUN-26
2	Mg Mg	HR	600000	ACTIVE	21-JUN-26	

```
SELECT * FROM audit_logs;
```

4 EMPLOYEES	
LOG_ID	TABLE_NAME

ACTION_NAME	RECORD_ID

OLD_VALUE	

NEW_VALUE	

ACTION_DA	

ACTION_USER	

DELETE	3

LOG_ID	TABLE_NAME

ACTION_NAME	RECORD_ID

OLD_VALUE	

NEW_VALUE	

ACTION_DA	

ACTION_USER	

Name=Su Su, Dept=Finance, Salary=1200000, Status=INACTIVE	

```
/* =====
10. STATEMENT LEVEL TRIGGER

Statement level trigger fires once per SQL statement.
It does NOT use FOR EACH ROW.
It cannot access :OLD and :NEW.
===== */

CREATE OR REPLACE TRIGGER trg_emp_statement_update
AFTER UPDATE ON employees
BEGIN
    DBMS_OUTPUT.PUT_LINE('STATEMENT LEVEL TRIGGER: Employee update
statement executed. ');
END;
/
```

```

/* =====
Testing UPDATE EMPLOYEE
This UPDATE will fire:
- BEFORE UPDATE row trigger
- AFTER UPDATE row trigger
- AFTER UPDATE statement trigger
===== */

```

```

SQL> UPDATE employees
SET salary = salary + 100000
WHERE employee_id = 2;
  2      3 BEFORE UPDATE Trigger: Employee update validated.
AFTER UPDATE Trigger: Employee audit log inserted.
STATEMENT LEVEL TRIGGER: Employee update statement executed.

1 row updated.

```

```
SQL> SELECT * FROM employees;
```

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
1	Aung Aung	IT	1000000	ACTIVE	21-JUN-26	21-JUN-26

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	STATUS	CREATED_D	UPDATED_D
2	Mg Mg	HR	700000	ACTIVE	21-JUN-26	21-JUN-26

```
COMMIT;
```

```

/* =====
14.
===== */

/* =====
LAB 1: AUDIT LOG TRIGGER

Requirement:
- Track INSERT, UPDATE, DELETE on customers table.
- Save old and new values.
===== */

```

```

CREATE OR REPLACE TRIGGER trg_customer_audit
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW
BEGIN

```

```

    IF INSERTING THEN
        INSERT INTO audit_logs
        (
            table_name,
            action_name,
            record_id,
            old_value,
            new_value,
            action_user
        )
        VALUES
        (
            'CUSTOMERS',
            'INSERT',
            :NEW.customer_id,
            NULL,
            'Name=' || :NEW.customer_name ||
            ', Phone=' || :NEW.phone ||
            ', City=' || :NEW.city ||
            ', Status=' || :NEW.status,
            USER
        );

```

```

    ELSIF UPDATING THEN
        INSERT INTO audit_logs
        (
            table_name,
            action_name,
            record_id,
            old_value,
            new_value,
            action_user
        )
        VALUES
        (
            'CUSTOMERS',
            'UPDATE',
            :NEW.customer_id,

```



```

        'Old Phone=' || :OLD.phone ||
        ', Old City=' || :OLD.city ||
        ', Old Status=' || :OLD.status,
        'New Phone=' || :NEW.phone ||
        ', New City=' || :NEW.city ||
        ', New Status=' || :NEW.status,
        USER
    );

ELSIF DELETING THEN
    INSERT INTO audit_logs
    (
        table_name,
        action_name,
        record_id,
        old_value,
        new_value,
        action_user
    )
    VALUES
    (
        'CUSTOMERS',
        'DELETE',
        :OLD.customer_id,
        'Name=' || :OLD.customer_name ||
        ', Phone=' || :OLD.phone ||
        ', City=' || :OLD.city ||
        ', Status=' || :OLD.status,
        NULL,
        USER
    );
END IF;
END;
/

/* Test Customer Audit Trigger */

INSERT INTO customers
(customer_id, customer_name, phone, city, status, created_date)
VALUES
(1, 'ABC Trading', '091111111', 'Yangon', 'ACTIVE', SYSDATE);

UPDATE customers
SET phone = '099999999',
    city = 'Mandalay'
WHERE customer_id = 1;

DELETE FROM customers
WHERE customer_id = 1;

COMMIT;

```

```

/* =====
LAB 2: SALARY HISTORY TRIGGER

Requirement:
- When employee salary changes, save old salary,
  new salary, and changed amount.
===== */

CREATE OR REPLACE TRIGGER trg_salary_history
AFTER UPDATE OF salary ON employees
FOR EACH ROW
WHEN (OLD.salary <> NEW.salary)
BEGIN
    INSERT INTO salary_history
    (
        employee_id,
        old_salary,
        new_salary,
        changed_amount,
        changed_by
    )
    VALUES
    (
        :NEW.employee_id,
        :OLD.salary,
        :NEW.salary,
        :NEW.salary - :OLD.salary,
        USER
    );

    DBMS_OUTPUT.PUT_LINE('Salary History Trigger: Salary change
recorded. ');
END;
/

/* Test Salary History Trigger */

UPDATE employees
SET salary = salary + 50000
WHERE employee_id = 1;

UPDATE employees
SET salary = salary + 75000
WHERE employee_id = 2;

COMMIT;

```

```

/* =====
LAB 3: CUSTOMER ACTIVITY TRACKING

Requirement:
- Track customer insert, update, delete activities.
- Store activity description.
===== */

CREATE OR REPLACE TRIGGER trg_customer_activity
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO customer_activity_logs
        (
            customer_id,
            activity_type,
            description,
            activity_user
        )
        VALUES
        (
            :NEW.customer_id,
            'CUSTOMER_CREATED',
            'Customer ' || :NEW.customer_name || ' was created.',
            USER
        );

    ELSIF UPDATING THEN
        INSERT INTO customer_activity_logs
        (
            customer_id,
            activity_type,
            description,
            activity_user
        )
        VALUES
        (
            :NEW.customer_id,
            'CUSTOMER_UPDATED',
            'Customer updated from Phone=' || :OLD.phone ||
            ' to Phone=' || :NEW.phone ||
            ', City=' || :OLD.city ||
            ' to City=' || :NEW.city,
            USER
        );

    ELSIF DELETING THEN
        INSERT INTO customer_activity_logs
        (
            customer_id,
            activity_type,
            description,
            activity_user

```

```

        )
VALUES
(
    :OLD.customer_id,
    'CUSTOMER_DELETED',
    'Customer ' || :OLD.customer_name || ' was deleted.',
    USER
);
END IF;
END;
/

/* Test Customer Activity Tracking */

INSERT INTO customers
(customer_id, customer_name, phone, city, status, created_date)
VALUES
(2, 'Global Tech', '092222222', 'Yangon', 'ACTIVE', SYSDATE);

UPDATE customers
SET phone = '098888888',
    city = 'Naypyidaw'
WHERE customer_id = 2;

DELETE FROM customers
WHERE customer_id = 2;

COMMIT;

```

```

/* =====
FINAL CHECKING
===== */

```

```
SQL> SELECT * FROM employees ORDER BY employee_id;
```

```

EMPLOYEE_ID
-----
EMPLOYEE_NAME
-----
DEPARTMENT                                SALARY
-----
STATUS          CREATED_D  UPDATED_D
-----
          1
Aung Aung
IT                                1000000
ACTIVE          21-JUN-26  21-JUN-26

```

```

EMPLOYEE_ID
-----
EMPLOYEE_NAME
-----
DEPARTMENT                                SALARY
-----
STATUS          CREATED_D  UPDATED_D
-----
          2
Mg Mg
HR                                700000
ACTIVE          21-JUN-26  21-JUN-26

```

```
SQL> SELECT * FROM customers ORDER BY customer_id;
```

```
no rows selected
```

```
SELECT * FROM audit_logs ORDER BY log_id;
```

```
ACTION_NAME                                RECORD_ID
-----
OLD_VALUE
-----
NEW_VALUE
-----
ACTION_DA
-----
ACTION_USER
-----
21-JUN-26
LOG_ID TABLE_NAME
-----
ACTION_NAME                                RECORD_ID
-----
OLD_VALUE
-----
NEW_VALUE
-----
ACTION_DA
-----
ACTION_USER
-----
SYSTEM
LOG_ID TABLE_NAME
-----
ACTION_NAME                                RECORD_ID
-----
OLD_VALUE
-----
NEW_VALUE
-----
ACTION_DA
-----
ACTION_USER
-----

11 rows selected.
```

```
SQL> SELECT * FROM salary_history ORDER BY history_id;

no rows selected
```

```
SQL> SELECT * FROM customer_activity_logs ORDER BY activity_id;
```

```
ACTIVITY_ID CUSTOMER_ID ACTIVITY_TYPE
```

```
-----  
DESCRIPTION
```

```
-----  
ACTIVITY_  
-----
```

```
ACTIVITY_USER
```

```
-----  
1 2 CUSTOMER_CREATED  
Customer Global Tech was created.  
21-JUN-26  
SYSTEM
```

```
ACTIVITY_ID CUSTOMER_ID ACTIVITY_TYPE
```

```
-----  
DESCRIPTION
```

```
-----  
ACTIVITY_  
-----
```

```
ACTIVITY_USER
```

```
-----  
2 2 CUSTOMER_UPDATED  
Customer updated from Phone=092222222 to Phone=098888888, City=Yangon to City=Na  
ypyidaw  
21-JUN-26  
SYSTEM
```

```
ACTIVITY_ID CUSTOMER_ID ACTIVITY_TYPE
```

```
-----  
DESCRIPTION
```

```
-----  
ACTIVITY_  
-----
```

```
ACTIVITY_USER
```

```
-----  
3 2 CUSTOMER_DELETED  
Customer Global Tech was deleted.  
21-JUN-26  
SYSTEM
```